



ENSTA PARIS

Institut Polytechnique de Paris

Research Project — *Projet de Recherche* (PRe)

Specialisation: Applied Mathematics

Academic year: 2025–2026

Promotion: 2027

Neural Network Acceleration of a Forward–Backward Splitting Scheme

for TGV^2 Tomographic Reconstruction

Author:

Théo Beille

Host supervisor:

Prof. Ozan ÖKTEM

ENSTA Paris tutor:

Prof. Andrea Simonetto

Internship carried out from 15/01/2026 to 06/06/2026

Host organisation: KTH Royal Institute of Technology (Department of Mathematics)

Lindstedtsvägen 25, SE-100 44 Stockholm, Sweden

Non confidential Report

Résumé

Ce rapport s'inscrit dans le cadre du projet de recherche de 2^{ème} année à l'ENSTA. Celui-ci présente l'étude de l'accélération de la convergence d'un algorithme itératif pour la résolution du problème de reconstruction tomographique régularisé par la variation généralisée totale d'ordre deux (TGV²). Le TGV² est un excellent modèle d'image, mais son principal défaut est son coût de calcul. L'objectif est de mesurer le niveau de précision atteignable en peu d'itérations (une dizaine) grâce à l'apprentissage, afin de contourner ce goulot d'étranglement. Le problème est d'abord reformulé comme une inclusion monotone $0 \in \mathcal{A}x + \mathcal{C}x$ dans un espace de Hilbert. Puis cette inclusion est résolue grâce à un algorithme itératif. $\mathcal{C}$ et $\mathcal{A}$ sont deux opérateurs devant respecter certaines conditions pour que l'algorithme puisse être appliqué.

Nous utilisons un schéma Forward–Backward Splitting (FBS) récent muni de déviations dépendant de l'historique, dont la convergence est assurée par une condition de sauvegarde : pour toute suite de déviations admissible, les itérés convergent vers une solution. Plutôt que de choisir ces déviations à la main, nous entraînons un réseau de neurones convolutif léger à prédire, à chaque itération, la déviation qui accélère le plus la convergence, une sauvegarde stricte renormalisant sa sortie de sorte que la garantie théorique soit toujours préservée. Sur des fantômes de tomographie à faisceau parallèle, le schéma appris atteint une précision donnée en nettement moins d'itérations que la ligne de base sans déviation et que la méthode PDHG, le gain restant valable lors d'un transfert $128 \times 128 \rightarrow 512 \times 512$ sans ré-entraînement.

Mots-clés : reconstruction tomographique, variation généralisée totale, éclatement avant-arrière, inclusion monotone, optimisation déroulée, apprentissage profond.

Abstract

This report was carried out as the second-year research project (Pre) at ENSTA. It studies how to make second-order total generalised variation (TGV²) tomographic reconstruction usable in practice. TGV² is an excellent image model, but its major drawback is its computational cost: the resulting optimisation is non-smooth and coupled, and standard first-order solvers need many iterations to converge. Our aim is to measure what level of accuracy can be reached in only a few iterations (about ten) by learning, in order to bypass this bottleneck.

To this end, the reconstruction problem is first recast as a monotone inclusion $0 \in \mathcal{A}x + \mathcal{C}x$ in a Hilbert space, where $\mathcal{C}$ and $\mathcal{A}$ are two operators that must satisfy certain conditions for the algorithm to be applicable. Then we use a recent forward–backward splitting (FBS) scheme with history-dependent deviations, whose convergence is guaranteed by a Lyapunov safeguarding condition: for any admissible deviation sequence the iterates converge to a solution. Rather than selecting the deviations by hand, we train a lightweight convolutional network to predict, at each iteration, the deviation that most accelerates convergence, while a hard safeguard rescales its output so that the theoretical guarantee is never lost.

On parallel-beam computed-tomography phantoms, the learned scheme reaches a given reconstruction accuracy in substantially fewer iterations than both the zero-deviation baseline and the standard primal–dual hybrid gradient (PDHG) method, with most of the gain concentrated in the first ten iterations. A random-direction control confirms that the gain is specific to *learning* rather than to perturbation alone, the safeguard keeps the learned iteration convergent far beyond its training window, and the fully convolutional architecture transfers, without retraining, from 128×128 to 512×512 problems.

Keywords: tomographic reconstruction, total generalised variation, forward–backward splitting, monotone inclusion, unrolled optimisation, deep learning, convergence safeguarding.

Remerciements

I would like to express my sincere gratitude to Prof. Ozan Öktem and Paul Hausner for welcoming me into their group at KTH and for giving me the opportunity to work on such a stimulating subject. This internship was my first real experience in deep learning, and their guidance and time throughout were more than useful.

Contents

Résumé	i
Abstract	ii
Remerciements	iii
1 Introduction and Context	1
1.1 Computed tomography as an inverse problem	1
1.2 From total variation to TGV ²	1
1.3 Iterative solvers and their limitations	2
1.4 Learning to accelerate the convergence of this problem	2
2 The research project	3
3 Mathematical Formulation	4
3.1 The TGV ² reconstruction problem	4
3.2 Saddle-point formulation	4
3.3 Monotone inclusion structure	5
3.4 From denoising to tomography: a single switch	7
4 Forward–Backward Splitting with Deviations	7
4.1 The algorithm	7
4.2 The safeguarding condition	9
4.3 Computing the resolvent	9
5 Learning the Deviations	12
5.1 Learning by unrolling the iteration	12
5.2 Network architecture	13
5.3 From raw output to a safeguarded deviation	14
5.4 Training	15
6 Experiments and Results	15
6.1 Experimental setup	15
6.2 Step-size selection on the baseline	18
6.3 Choosing the training target: inclusion residual versus objective	18

6.3.1	Training on the KKT residual	19
6.3.2	Does it also lower the objective? Comparison with PDHG	20
6.3.3	Training on the objective directly	22
6.4	Reconstruction quality and PSNR	24
6.5	Training behaviour	25
6.6	Generalisation across resolution	26
6.7	Discussion	26
7	Conclusion and Perspectives	27
7.1	Summary	27
7.2	Perspectives	27

Chapter 1

Introduction and Context

1.1 Computed tomography as an inverse problem

Computed tomography (CT) reconstructs an unknown image $u : \Omega \rightarrow \mathbb{R}$, defined on a bounded domain $\Omega \subset \mathbb{R}^2$, from indirect measurements of it. The acquisition is modelled by a linear forward operator K ,

$$y = Ku + \varepsilon, \quad (1.1)$$

where K maps the image to the measurements, y is the measured data and ε is measurement noise. The exact form of K depends on the imaging geometry and is an implementation choice. Everything that follows is written for a generic measurement operator K that is supposed to be linear, and the concrete operator we use is specified later (Chapter 6). Recovering u from y is a classical ill-posed inverse problem [3]: the operator K typically has a non-trivial null space, so a naive inversion produces bad reconstructions. The standard remedy is variational regularisation: instead of inverting K directly, one minimises a sum of a data-fidelity term and a regulariser $\mathcal{R}(u)$,

$$\min_u \frac{1}{2} \|Ku - y\|^2 + \mathcal{R}(u). \quad (1.2)$$

1.2 From total variation to TGV²

One of the most popular regularisers is *total variation* (TV) [4], $\mathcal{R}(u) = \|\nabla u\|_1$, which penalises the image gradient and therefore favours piecewise-constant images with sharp edges. TV is remarkably effective at removing noise while preserving discontinuities, but it has a well-known defect: because it rewards piecewise-constant solutions, it turns smooth surfaces into flat plateaus separated by jumps. This is the staircasing effect, and it is particularly visible in medical images, where soft-tissue gradients are quite common.

Total generalised variation of order two, TGV², introduced by Bredies, Kunisch and Pock [5, 6], repairs this defect by penalising not only the gradient but also its (symmetrised) derivative. Smooth ramps are no longer penalised while edges are still preserved. It does so by introducing an auxiliary vector field w that plays the role of the gradient where the image is smooth:

$$\text{TGV}_{\alpha_1, \alpha_2}^2(u) = \min_w \alpha_1 \|\nabla u - w\|_1 + \alpha_2 \|\mathcal{E}w\|_1, \quad (1.3)$$

where $\mathcal{E}w = \frac{1}{2}(\nabla w + (\nabla w)^\top)$ is the symmetrised gradient and $\alpha_1, \alpha_2 > 0$ balance the first- and second-order terms. Therefore it has great interest for the scientific field but the price to pay for this superior image model is a more complex, *non-smooth* and *coupled* optimisation problem which is one of the main reasons it was not developed in industries like the medical field. The current methods are too slow to be of interest.

1.3 Iterative solvers and their limitations

Because the TGV^2 objective is convex but non-smooth, it is solved by first-order splitting methods. In this internship a reference algorithm has been used: the primal–dual hybrid gradient method (PDHG, also known as Chambolle–Pock) [7], which converges at rate $O(1/t)$ in the objective. This rate, combined with a cost per iteration that grows with the image size, is the practical bottleneck: realistic CT volumes reach 512×512 pixels per slice or more, and an $O(1/t)$ method may need hundreds of iterations to reach an acceptable accuracy (prohibitive for real-time or high-throughput acquisition). Accelerated schemes such as FISTA [8] attain $O(1/t^2)$, but only for smooth objectives. However, the TGV^2 functional is non-smooth, so these accelerations do not apply directly.

1.4 Learning to accelerate the convergence of this problem

Recent work combines classical splitting algorithms with deep learning, keeping the convergence guarantees of one while borrowing the empirical speed of the other. This is an active direction: a closely related work also unrolls a second-order TGV^2 CT reconstruction into a network [9], though on Chambolle–Pock rather than on the safeguarded scheme used here. Our work rests on two ideas. The first is the *forward–backward splitting with history and deviations* of Sadeghi, Banert and Giselsson [1]: the iterates may be perturbed at each step by deviations (Δ_n^1, Δ_n^2) , and as long as these stay inside a safeguarding set, convergence is guaranteed for any admissible sequence. The deviations thus become a free steering direction that can be chosen aggressively without endangering convergence, the perfect substrate for learning. The second is *algorithm unrolling* [10, 2]: a fixed number of iterations is treated as the layers of a network whose free parameters are learned end-to-end.

The aim is therefore to formulate the TGV^2 problem as a monotone inclusion, so as to fall within the study case of [1], and to accelerate it by learning the deviations. Concretely, what we want to measure is the accuracy reachable in only a few iterations (about ten), the regime that matters in practice: the obstacle to using TGV^2 is its computational cost, and reaching a good reconstruction within a handful of iterations is exactly what would remove it. Running the

iteration far beyond this horizon only confirms that the safeguarded scheme stays convergent.

Chapter 2

The research project

At the start of the internship the project was only a theoretical opportunity to combine two published lines of work: the forward–backward splitting scheme with history and deviations of Sadeghi, Banert and Giselsson [1], and the deep-learning acceleration idea of Banert et al. [2], demonstrated there on a different problem class. No instantiation existed for TGV² tomography: no monotone-inclusion formulation, no implementation, no resolvent solver, no learned-deviation network. Everything in Chapters 3–6 was therefore built from scratch during the internship, the only component reused as-is being the convergence theorem of [1]. For the learning side I largely inspired myself with the network and training strategy of [2]. My personal contributions are:

1. a reformulation of the TGV² CT problem as a monotone inclusion $0 \in \mathcal{A}x + \mathcal{C}x$, with $\mathcal{A}$ split into a skew-symmetric coupling and the normal cones of the dual constraints and $\mathcal{C}$ the cocoercive data gradient (Chapter 3).
2. an implementation of the FBS-with-deviations algorithm, including the inner FISTA resolvent (Chapter 4).
3. a lightweight convolutional deviation network with a normalisation-and-safeguard layer that provably keeps the iteration convergent whatever the network outputs (Chapter 5).
4. an experimental study (Chapter 6): the learned deviations beat both the zero-deviation baseline and PDHG, a random-direction control attributes the gain to learning, the safeguard preserves convergence far beyond the training horizon, and the network transfers from 128×128 to 512×512 without retraining.

The python code of the project can be found on: github.com/TheoBeille/Research-Internship.

Chapter 3

Mathematical Formulation

This chapter turns the TGV² reconstruction problem into the monotone inclusion that the rest of the report solves. The path is: variational problem $\rightarrow$ joint minimisation $\rightarrow$ saddle-point problem $\rightarrow$ optimality system $\rightarrow$ monotone inclusion.

3.1 The TGV² reconstruction problem

Let $\Omega \subset \mathbb{R}^2$ be the image domain discretised on an $N \times N$ grid, so that images live in $U \simeq \mathbb{R}^{N^2}$. We further introduce the product spaces

$$V := U \times U, \quad X := V \times V,$$

which host the auxiliary field w and the symmetrised gradient $\mathcal{E}w$ respectively. The TGV² reconstruction problem reads

$$\min_{u \in \mathbb{R}^{N^2}} \frac{1}{2} \|Ku - y\|^2 + \text{TGV}_{\alpha_1, \alpha_2}^2(u), \quad (3.1)$$

with the functional defined, as in (1.3), by an inner minimisation over the auxiliary field w . Here ∇ is the discrete forward-difference gradient, $\mathcal{E}w = \frac{1}{2}(\nabla w + (\nabla w)^\top)$ is the symmetrised gradient, and the ℓ^1 -type norm is $\|\cdot\|_1 := \sup_{\|p\|_\infty \leq 1} \langle p, \cdot \rangle$.

Substituting the definition (1.3) into (3.1) and viewing w as an independent optimisation variable turns the problem into a single joint minimisation,

$$\min_{u, w} \frac{1}{2} \|Ku - y\|^2 + \alpha_1 \|\nabla u - w\|_1 + \alpha_2 \|\mathcal{E}w\|_1. \quad (3.2)$$

3.2 Saddle-point formulation

The two non-smooth ℓ^1 terms are handled by dualisation. We introduce dual variables p (conjugate to $\nabla u - w$, constrained by $\|p\|_\infty \leq \alpha_1$) and q (conjugate to $\mathcal{E}w$, constrained by $\|q\|_\infty \leq \alpha_2$). Problem (3.2) becomes the convex-concave saddle-point problem

$$\min_{u, w} \max_{p, q} \frac{1}{2} \|Ku - y\|^2 + \langle p, \nabla u - w \rangle + \langle q, \mathcal{E}w \rangle - \iota_{\|p\|_\infty \leq \alpha_1}(p) - \iota_{\|q\|_\infty \leq \alpha_2}(q), \quad (3.3)$$

where ι_C denotes the convex indicator of a set C (zero on C , $+\infty$ elsewhere). The optimality conditions $0 \in \partial \mathcal{L}(u, w, p, q)$ of the Lagrangian $\mathcal{L}$ expand to the system

$$\begin{cases} 0 = K^\top(Ku - y) + \nabla^\top p, \\ 0 = -p + \mathcal{E}^\top q, \\ 0 \in (\nabla u - w) + \partial \iota_{\|p\|_\infty \leq \alpha_1}(p), \\ 0 \in \mathcal{E}w + \partial \iota_{\|q\|_\infty \leq \alpha_2}(q). \end{cases} \quad (3.4)$$

3.3 Monotone inclusion structure

Collecting the four variables into $x = (u, w, p, q)^\top$ in the product Hilbert space

$$\mathcal{H} = L^2(\Omega) \times L^2(\Omega)^2 \times L^2(\Omega)^2 \times L^2(\Omega)^3,$$

the system (3.4) is equivalent to the *monotone inclusion*

$$0 \in \mathcal{A}x + \mathcal{C}x, \quad (3.5)$$

where the two operators are defined as follows.

The operator $\mathcal{C}$. All the measured data enter through the gradient of the data-fidelity term:

$$\mathcal{C}x = (K^\top(Ku - y), 0, 0, 0)^\top = \nabla_u \left(\frac{1}{2} \|Ku - y\|^2 \right). \quad (3.6)$$

The operator $\mathcal{A}$. The remaining terms assemble into a maximally monotone operator that splits as $\mathcal{A} = \mathcal{A}_{\text{skew}} + \mathcal{A}_{\text{prox}}$, with

$$\mathcal{A}_{\text{skew}} = \begin{pmatrix} 0 & 0 & \nabla^\top & 0 \\ 0 & 0 & -\text{I} & \mathcal{E}^\top \\ -\nabla & \text{I} & 0 & 0 \\ 0 & -\mathcal{E} & 0 & 0 \end{pmatrix}, \quad (3.7)$$

which is skew-symmetric, and

$$\mathcal{A}_{\text{prox}}x = (0, 0, \partial \iota_{\|p\|_\infty \leq \alpha_1}(p), \partial \iota_{\|q\|_\infty \leq \alpha_2}(q))^\top, \quad (3.8)$$

the normal-cone operator of the dual constraint sets. Summing (3.7) and (3.8) gives the explicit form

$$\mathcal{A}x = \begin{pmatrix} \nabla^\top p \\ -p + \mathcal{E}^\top q \\ -(\nabla u - w) + \partial \iota_{\|p\|_\infty \leq \alpha_1}(p) \\ -\mathcal{E}w + \partial \iota_{\|q\|_\infty \leq \alpha_2}(q) \end{pmatrix}. \quad (3.9)$$

For a closed convex set C the normal cone and the projection are linked by $(\text{I} + \partial \iota_C)^{-1} = \text{proj}_C$, so it is the projection that appears in practice. The corresponding projection is computed

independently at each pixel i :

$$(\text{proj}_{\|\cdot\|_\infty \leq \alpha}(z))_i = \frac{z_i}{\max(1, |z_i|/\alpha)} = \begin{cases} z_i, & |z_i| \leq \alpha, \\ \alpha \frac{z_i}{|z_i|}, & |z_i| > \alpha, \end{cases} \quad (3.10)$$

where $|z_i|$ denotes the Euclidean norm of the (vector-valued) field z at pixel i . In words, $\text{proj}_{\|\cdot\|_\infty \leq \alpha}$ leaves a value untouched when it already lies inside the ball of radius α and otherwise rescales it back onto the sphere of radius α .

Operator structure required by the algorithm. To apply the forward–backward scheme of [1], the two operators must respect the monotone structure for which that scheme is proved to converge: the forward operator $\mathcal{C}$ must be $\frac{1}{\beta}$ -cocoercive for some $\beta > 0$, and the backward operator $\mathcal{A}$ must be maximally monotone. The rest of this section verifies each of these two properties in turn (Proposition 3.1 for $\mathcal{C}$ and Proposition 3.2 for $\mathcal{A}$).

Proposition 3.1 (Cocoercivity of $\mathcal{C}$). *Let $\beta = \|K\|^2$. The operator $\mathcal{C}$ in (3.6) is $\frac{1}{\beta}$ -cocoercive, i.e. for all $x, x' \in \mathcal{H}$,*

$$\langle \mathcal{C}x - \mathcal{C}x', x - x' \rangle \geq \frac{1}{\beta} \|\mathcal{C}x - \mathcal{C}x'\|^2. \quad (3.11)$$

Proof. We assume that K is linear in our case. Write $f(x) = \frac{1}{2} \|Ku - y\|^2$, a convex quadratic in $x = (u, w, p, q)$ depending only on the u -block. Then $\mathcal{C} = \nabla f$ is the gradient of f . Its Jacobian is the constant, symmetric positive-semidefinite operator $\text{diag}(K^\top K, 0, 0, 0)$, so $\mathcal{C}$ is affine and monotone, and it is Lipschitz with constant $\|K^\top K\| = \|K\|^2 = \beta$:

$$\|\mathcal{C}x - \mathcal{C}x'\| = \|K^\top K(u - u')\| \leq \|K\|^2 \|u - u'\| \leq \beta \|x - x'\|.$$

Since f is convex and differentiable with a β -Lipschitz gradient, [11, Cor. 18.16] states that ∇f is $\frac{1}{\beta}$ -cocoercive, which is exactly (3.11). $\square$

Proposition 3.2 (Maximal monotonicity of $\mathcal{A}$). *The operator $\mathcal{A} = \mathcal{A}_{\text{skew}} + \mathcal{A}_{\text{prox}}$ defined in (3.7)–(3.8) is maximally monotone.*

Proof. We treat the two summands separately. $\mathcal{A}_{\text{skew}}$ is a bounded linear operator with $\mathcal{A}_{\text{skew}}^\top = -\mathcal{A}_{\text{skew}}$, so $\langle \mathcal{A}_{\text{skew}}x, x \rangle = 0$ for every x (in particular it is monotone), and being everywhere defined and continuous it is maximally monotone [11, Ex. 20.35]. $\mathcal{A}_{\text{prox}} = \partial \iota_C$ is the subdifferential of the indicator ι_C of the closed convex constraint set $C = \{\|p\|_\infty \leq \alpha_1\} \times \{\|q\|_\infty \leq \alpha_2\}$ (a proper, lower semicontinuous, convex function). The subdifferential of such a function is maximally monotone [11, Thm. 20.25]. Finally, $\mathcal{A}_{\text{skew}}$ is maximally monotone with full domain $\text{dom } \mathcal{A}_{\text{skew}} = \mathcal{H}$, so the sum rule for maximally monotone operators [11, Cor. 25.5] applies and $\mathcal{A} = \mathcal{A}_{\text{skew}} + \mathcal{A}_{\text{prox}}$ is maximally monotone. $\square$

Together, Propositions 3.1 and 3.2 certify that (3.5) has exactly the cocoercive-plus-maximally-monotone structure required by the forward–backward algorithm of the next chapter.

3.4 From denoising to tomography: a single switch

Because the whole construction is written in terms of the abstract operator K , the same code solves two problems at once: $K = \text{I}$ gives the denoising problem (data gradient $\mathcal{C} = u - y$, operator norm $\|K\| = 1$), and K the measurement operator gives tomography (the Radon transform, in ODL). We deliberately started with denoising, where the trivial data operator and benign step-size regime made it the ideal place to develop and debug the whole pipeline (inclusion, resolvent, safeguard and learned deviations), before switching K to the tomographic operator.

Chapter 4

Forward–Backward Splitting with Deviations

We now recall the algorithm of Sadeghi, Banert and Giselsson [1] that solves the inclusion (3.5), state its safeguarding condition and convergence theorem, and explain how the resolvent of $\mathcal{A}$ is computed.

4.1 The algorithm

Algorithm 1 solves a monotone inclusion $0 \in \mathcal{A}x + \mathcal{C}x$ with $\mathcal{A}$ maximally monotone and $\mathcal{C}$ cocoercive (exactly the structure derived in Chapter 3). Its distinguishing feature is the pair of deviations (Δ_n^1, Δ_n^2) injected at each iteration.

This work focuses on *learning the deviations*, not on choosing the parameter sequences. A big freedom lay in this choice: many admissible choices satisfy the hypotheses below, and some could make the algorithm converge faster than others. One could in principle learn them on the same idea as the deviations (a next step left unexplored for lack of time). We therefore place ourselves in a particular admissible family studied in [1], which is known to satisfy the parameter conditions, and concentrate on the effect of the deviations.

Consider the non-decreasing positive sequences

$$\lambda_n = \lambda_0(1+n)^e, \quad \mu_n = \frac{1}{\lambda_0}\lambda_n^2 - \lambda_n,$$

parameterised by a scale $\lambda_0 > 0$ and an exponent $e \in [0, 1]$, whose concrete values are fixed in Chapter 6. The following conditions on the parameters then ensure convergence (proof in [1]).

Algorithm 1 Forward–Backward Splitting with History and Deviations [1]

Requirements: initial point $x_0 \in \mathcal{H}$; sequences $(\gamma_n), (\lambda_n), (\mu_n), (\zeta_n)$ and constants $\bar{\beta} \geq \beta \geq 0$ satisfying Assumption 4.1.

1: $y_{-1} = p_{-1} = z_{-1} = x_0$; $\Delta_0^1 = \Delta_0^2 = 0$; $\gamma_{-1} = \gamma_0$

2: **for** $n = 0, 1, 2, \dots$ **do**

Define the parameters

$$\alpha_n = \frac{\mu_n}{\lambda_n + \mu_n}, \quad \bar{\alpha}_n = \frac{\gamma_n \mu_n}{\gamma_{n-1}(\lambda_n + \mu_n)}, \quad \theta_n = (4 - \gamma_n \bar{\beta})(\lambda_n + \mu_n) - 2\lambda_n^2,$$

$$\hat{\theta}_n = 2(\lambda_n + \mu_n) - \gamma_n \bar{\beta} \lambda_n^2, \quad \bar{\theta}_n = \lambda_n + \mu_n - \lambda_n^2, \quad \tilde{\theta}_n = (\lambda_n + \mu_n) \gamma_n \bar{\beta}.$$

$$y_n \leftarrow x_n + \alpha_n(y_{n-1} - x_n) + \Delta_n^1$$

$$z_n \leftarrow x_n + \alpha_n(p_{n-1} - x_n) + \bar{\alpha}_n(z_{n-1} - p_{n-1}) + \frac{\bar{\theta}_n \gamma_n \bar{\beta}}{\theta_n} \Delta_n^1 + \Delta_n^2$$

$$p_n \leftarrow (I + \gamma_n \mathcal{A})^{-1}(z_n - \gamma_n \mathcal{C} y_n)$$

$$x_{n+1} \leftarrow x_n + \lambda_n(p_n - z_n) + \bar{\alpha}_n \lambda_n(z_{n-1} - p_{n-1})$$

Define:

$$\begin{aligned} \ell_n := & \frac{\theta_n}{2} \left\| p_n - x_n + \alpha_n(x_n - p_{n-1}) + \frac{\gamma_n \bar{\beta} \lambda_n^2}{\theta_n} \Delta_n^1 - \frac{2\bar{\theta}_n}{\theta_n} \Delta_n^2 \right\|^2 \\ & + 2\mu_n \gamma_n \left\langle \frac{z_n - p_n}{\gamma_n} - \frac{z_{n-1} - p_{n-1}}{\gamma_{n-1}}, p_n - p_{n-1} \right\rangle + \frac{\mu_n \gamma_n \bar{\beta}}{2} \|p_n - y_n - (p_{n-1} - y_{n-1})\|^2. \end{aligned} \quad (4.1)$$

Choose deviations $(\Delta_{n+1}^1, \Delta_{n+1}^2)$ subject to the safeguard

$$(\lambda_{n+1} + \mu_{n+1}) \left(\frac{\tilde{\theta}_{n+1}}{\theta_{n+1}} \|\Delta_{n+1}^1\|^2 + \frac{\hat{\theta}_{n+1}}{\theta_{n+1}} \|\Delta_{n+1}^2\|^2 \right) \leq \zeta_{n+1} \ell_n. \quad (4.2)$$

3: **end for**

Assumption 4.1 (Parameter conditions of [1]). There exist $\varepsilon > 0$ and $\varepsilon_0, \varepsilon_1 \geq 0$, $\lambda_0 > 0$, such that for all $n \in \mathbb{N}$:

- (1) $0 \leq \zeta_n \leq 1 - \varepsilon_0$;
- (2) $\varepsilon \leq \gamma_n \bar{\beta} \leq 4 - 2\lambda_0 - \varepsilon$;
- (3) $\lambda_{n+1} \geq \lambda_n$ and $\gamma_n \lambda_n - \gamma_{n-1} \lambda_{n-1} \leq \gamma_n \lambda_0 - \varepsilon_1$;
- (4) $\bar{\beta} \geq \beta$, where β is the cocoercivity constant of $\mathcal{C}$ (Proposition 3.1).

Remark 4.2. Setting $\Delta_n^1 = \Delta_n^2 = 0$ recovers the standard forward–backward iteration, which we use throughout as the *zero-deviation baseline* against which the learned deviations are compared.

4.2 The safeguarding condition

Convergence is enforced by requiring the deviations of the *next* step to lie within the budget ℓ_n generated by the current step. This is exactly the safeguard inequality (4.2) already stated inside Algorithm 1,

$$(\lambda_{n+1} + \mu_{n+1}) \left(\frac{\tilde{\theta}_{n+1}}{\theta_{n+1}} \|\Delta_{n+1}^1\|^2 + \frac{\hat{\theta}_{n+1}}{\theta_{n+1}} \|\Delta_{n+1}^2\|^2 \right) \leq \zeta_{n+1} \ell_n.$$

The number $\zeta_{n+1} \in [0, 1)$ controls how much of the available budget the deviations may consume. The work focused more on the directions of the deviations rather than their amplitude, as discussed later in the report.

Theorem 4.3 (Convergence, [1, Thm. 1]). *Let x^* be a solution of (3.5). If the sequences $(\gamma_n), (\lambda_n), (\mu_n)$ satisfy Assumption 4.1 and the deviations satisfy the safeguard (4.2) with $\zeta_{n+1} \in [0, 1)$, then the iterates x_n of Algorithm 1 converge weakly to a solution x^* .*

The practical consequence is decisive: *whatever* rule produces the deviations (a hand-designed heuristic, a random generator, or a neural network), as long as its output is rescaled to satisfy (4.2), Theorem 4.3 guarantees convergence. The learning problem is thereby decoupled from the convergence problem.

4.3 Computing the resolvent

Before speaking of learning the deviations, the algorithm was first coded to work with random deviations lying in the convergent set. No particular difficulty arises in this algorithm except the requirement of the resolvent of $\mathcal{A}$.

In fact each iteration requires the resolvent $R_{\mathcal{A}}(z) = (\mathbf{I} + \gamma\mathcal{A})^{-1}(z)$, which returns $x^* = (u^*, w^*, p^*, q^*)$ solving the inclusion $z \in (\mathbf{I} + \gamma\mathcal{A})(x^*)$.

Using the explicit block form of $\mathcal{A}$,

$$\mathcal{A}x = \begin{pmatrix} \nabla^\top p \\ -p + \mathcal{E}^\top q \\ -(\nabla u - w) + \partial\iota_{\|p\|_\infty \leq \alpha_1}(p) \\ -\mathcal{E}w + \partial\iota_{\|q\|_\infty \leq \alpha_2}(q) \end{pmatrix},$$

the inclusion splits into four blocks:

$$z_u = u^* + \gamma \nabla^\top p^*, \tag{4.3}$$

$$z_w = w^* + \gamma(-p^* + \mathcal{E}^\top q^*), \tag{4.4}$$

$$z_p \in p^* + \gamma\left(-(\nabla u^* - w^*) + \partial\iota_{\|p\|_\infty \leq \alpha_1}(p^*)\right), \tag{4.5}$$

$$z_q \in q^* + \gamma\left(-\mathcal{E}w^* + \partial\iota_{\|q\|_\infty \leq \alpha_2}(q^*)\right). \tag{4.6}$$

The first two (primal) equations are linear. The last two (dual) equations invert through a projection: since the resolvent of an indicator's subdifferential is the metric projection [12], $(\mathbf{I} + \gamma \partial\iota_C)^{-1} = \text{proj}_C$ (the factor $\gamma > 0$ plays no role, $\partial\iota_C = N_C$ being a cone with $\gamma N_C = N_C$), so

$$p^* = \text{proj}_{\|\cdot\|_\infty \leq \alpha_1}(z_p + \gamma(\nabla u^* - w^*)), \quad q^* = \text{proj}_{\|\cdot\|_\infty \leq \alpha_2}(z_q + \gamma\mathcal{E}w^*).$$

Collecting, the resolvent amounts to solving the stationarity system:

$$(u^\star - z_u) + \gamma \nabla^\top p^\star = 0, \quad (4.7)$$

$$(w^\star - z_w) + \gamma(-p^\star + \mathcal{E}^\top q^\star) = 0, \quad (4.8)$$

$$p^\star = \text{proj}_{\|\cdot\|_\infty \leq \alpha_1}(z_p + \gamma(\nabla u^\star - w^\star)), \quad (4.9)$$

$$q^\star = \text{proj}_{\|\cdot\|_\infty \leq \alpha_2}(z_q + \gamma \mathcal{E} w^\star). \quad (4.10)$$

For TGV² this system has no closed form, so it is solved by an inner iterative solver. We present it in the order we discovered it: a first natural method, the limit that made it unusable, and the FISTA formulation that resolved both.

A first method: fixed-point iteration. The most natural idea is to read the resolvent as a fixed-point equation and to iterate it directly.

Group the primal variables as $\phi = (u, w)$ and the duals as $\xi = (p, q)$, and let B be the coupling operator

$$B\phi = (\nabla u - w, \mathcal{E}w), \quad B^\top \xi = (\nabla^\top p, -p + \mathcal{E}^\top q),$$

so that the stationarity system (4.7)–(4.10) can be written compactly as

$$\phi^\star + \gamma B^\top \xi^\star = z_\phi, \quad (4.11)$$

$$\xi^\star = \text{proj}_C(z_\xi + \gamma B\phi^\star), \quad (4.12)$$

with $C = \{\|p\|_\infty \leq \alpha_1\} \times \{\|q\|_\infty \leq \alpha_2\}$.

The primal equation (4.11) is linear and solved in closed form,

$$\phi^\star = z_\phi - \gamma B^\top \xi^\star. \quad (4.13)$$

Substituting the first into the second eliminates the primal variables and leaves a fixed-point equation in the dual variable alone,

$$\xi^\star = \text{proj}_C(b - \gamma^2 BB^\top \xi^\star), \quad b = z_\xi + \gamma B z_x. \quad (4.14)$$

The most natural solver simply iterates this map,

$$\xi^{k+1} = \text{proj}_C(b - \gamma^2 BB^\top \xi^k). \quad (4.15)$$

Convergence. This map is the composition of the nonexpansive projection proj_C with the affine map $\xi \mapsto b - \gamma^2 BB^\top \xi$, whose linear part has operator norm $\gamma^2 \|B\|^2$. The iteration is therefore a contraction, and converges, if and only if

$$\gamma^2 \|B\|^2 < 1 \quad \Longleftrightarrow \quad \gamma \|B\| < 1.$$

Its limit. In our experiments $\|B\| \approx 1.15$ (128×128 grid), so $\gamma < 0.78$. This caps the outer step precisely when one would like γ as large as the cocoercivity condition $\gamma\bar{\beta} \leq 4 - 2\lambda_0$ allows: the resolvent and the outer iteration fight over the same parameter, the inner contraction being the limiting one. This was tolerable for denoising, where a small step already converges well, but too restrictive for tomography.

The fix: FISTA on the resolvent’s saddle problem. The way out is to stop relying on the contraction and instead solve the resolvent’s own saddle problem with FISTA, whose convergence does not require $\gamma\|B\| < 1$.

FISTA (*Fast Iterative Shrinkage-Thresholding Algorithm*) [8] is the accelerated version of the proximal-gradient method (ISTA) for a composite problem

$$\min_{\xi} f(\xi) + g(\xi),$$

where f is smooth with an L -Lipschitz gradient and g is non-smooth but prox-friendly.

Algorithm 2 FISTA for the resolvent’s dual problem $\min_{\xi} f(\xi) + \iota_C(\xi)$

Require: smooth part f with L -Lipschitz gradient; initial ξ_0 .

```

1:  $y_1 \leftarrow \xi_0$ ;  $t_1 \leftarrow 1$ 
2: for  $k = 1, 2, \dots$  do
     $\xi_k \leftarrow \text{prox}_g(y_k - \frac{1}{L} \nabla f(y_k))$ 
     $t_{k+1} \leftarrow \frac{1 + \sqrt{1 + 4t_k^2}}{2}$ 
     $y_{k+1} \leftarrow \xi_k + \frac{t_k - 1}{t_{k+1}} (\xi_k - \xi_{k-1})$ 
3: end for
```

Return: ξ_k .

Finding f and g . Keep ϕ , ξ and B as before. The system (4.7)–(4.10) is the optimality condition of the convex–concave saddle problem $\min_{\phi} \max_{\xi \in C} S$ with

$$S = \frac{1}{2} \|\phi - z_{\phi}\|^2 - \frac{1}{2} \|\xi\|^2 + \langle \xi, z_{\xi} \rangle + \gamma \langle \xi, B\phi \rangle, \quad (4.16)$$

which is the reverse construction to the dualisation of the objective function of the initial TGV² problem. S is strongly convex in ϕ and concave in ξ , so by strong duality $\min_{\phi} \max_{\xi \in C} S = \max_{\xi \in C} (\min_{\phi} S)$. The inner minimiser in ϕ is explicit from (4.7)–(4.8):

$$\nabla_{\phi} S = (\phi - z_{\phi}) + \gamma B^{\top} \xi = 0 \implies \phi^* = z_{\phi} - \gamma B^{\top} \xi,$$

i.e. $u^* = z_u - \gamma \nabla^{\top} p$ and $w^* = z_w + \gamma(p - \mathcal{E}^{\top} q)$. Substituting back and using $\frac{1}{2} \|\phi^* - z_{\phi}\|^2 = \frac{\gamma^2}{2} \|B^{\top} \xi\|^2$ and $\gamma \langle \xi, B\phi^* \rangle = \gamma \langle \xi, Bz_{\phi} \rangle - \gamma^2 \|B^{\top} \xi\|^2$, the dual function depends on ξ only and is a concave quadratic,

$$d(\xi) = -\frac{1}{2} \langle \xi, (I + \gamma^2 B B^{\top}) \xi \rangle + \langle \xi, b \rangle, \quad b = z_{\xi} + \gamma B z_{\phi}. \quad (4.17)$$

Maximising d over C is equivalent to the composite minimisation $\min_{\xi} f(\xi) + g(\xi)$ with $f = -d$ and $g = \iota_C$, where

$$f \text{ is smooth with } \nabla^2 f = \mathbf{I} + \gamma^2 BB^\top, \quad L = \|\nabla^2 f\| = 1 + (\gamma \|B\|)^2, \quad (4.18)$$

and $g = \iota_C$ is non-smooth with proximal operator $\text{prox}_g = \text{proj}_C$. The problem is therefore “smooth + prox-friendly” and we solve it with FISTA [8].

Remark 4.4 (Why the inner FISTA frees the outer step). The Lipschitz constant $L = 1 + (\gamma \|B\|)^2$ already absorbs the factor $\gamma \|B\|$. Consequently the inner FISTA converges for any γ , it does not require the usual condition $\gamma \|B\| < 1$. This is what allows the outer iteration to use a large step size (the concrete value is fixed in Chapter 6) without destabilising the resolvent. In practice the inner solver is run to a tight tolerance with a capped iteration budget.

Chapter 5

Learning the Deviations

Algorithm 1 now runs on images of a given size, and it converges for any admissible deviations, in particular for zero or random ones. But this is not where the interest lies: the real benefit of the deviations is to steer the iteration so that it converges in as few iterations as possible. The theory gives no recipe for choosing them, only the constraint of staying inside the safeguarding set, so we delegate this choice to a neural network. This chapter explains the network and training choices.

5.1 Learning by unrolling the iteration

Algorithm unrolling [10, 2] has recently shown strong results for learning with iterative algorithms, which motivates its use here. One fixes a finite horizon T , runs the safeguarded iteration for T steps, and treats the whole sequence as a single differentiable computation: each iteration becomes one layer, the algorithm state $s_n = (x_n, y_{n-1}, p_{n-1}, z_{n-1}, \Delta_{n-1}^{(1),(2)}, \mathcal{C}(y_n))$ is the signal passed from layer to layer, and the deviation network, reused at every step with the same weights θ , is the only trainable part. Writing Φ for one (fixed) forward–backward step and g_θ for the network-plus-safeguard map producing the deviations, one layer is

$$s_{n+1} = \Phi(s_n, g_\theta(s_n)) =: F_\theta(s_n), \quad (5.1)$$

and the unrolled model is the T -fold composition

$$s_T = (\underbrace{F_\theta \circ \dots \circ F_\theta}_{T \text{ times}})(s_0) = F_\theta^{\circ T}(s_0). \quad (5.2)$$

This is a weight-tied deep network, optimised end-to-end: a single loss on the final iterate x_T is back-propagated through all T layers to update θ (Section 5.4). In effect, training adjusts the whole trajectory $s_0 \rightarrow \dots \rightarrow s_T$ at once, tuning the steering rule for the state the iteration actually reaches.

5.2 Network architecture

The learned deviations are produced by a convolutional network $\mathcal{N}_\theta$ that maps the current algorithm state to raw deviations (h_n^1, h_n^2) , then normalised and safeguarded (Section 5.3). The guiding principle is that the network only produces a search direction, rescaled inside the convergent scheme, so the safeguard guarantees convergence for any output of $\mathcal{N}_\theta$.

The architecture is fully convolutional and acts only on the primal blocks, for two reasons. First, a fully connected network does not scale: the state carries eight scalar fields per pixel, so a dense input layer already has $8N^2$ inputs and a parameter count growing quadratically with the image size (hopeless at 128×128 , let alone 512×512). A convolutional network removes this dependence, the same weights acting at every pixel, so the parameter count is independent of the image size and the network transfers to a resolution it was never trained on (Chapter 6). Even so, propagating the full $8 \times N \times N$ state is memory-heavy, so we restrict the learned deviations to the *primal* blocks, $h_n = (*, *, 0, 0)$: this halves the output channels at no measurable cost in convergence, and the primal variables are the ones that carry the image. Second, the problem is ultimately about an image, and convolutional networks are the natural tool: by sharing small kernels across pixels they are translation-equivariant and exploit local spatial correlations, which is why they are standard for image problems [13, 14] and the building block of unrolled CT reconstruction networks with the same TGV structure [9].

We use the same lightweight backbone as classical unrolled reconstruction networks [2, 9].

Inputs. At iteration n the network receives the seven state quantities

$$(x_n, p_n, y_n, z_n, \Delta_{n-1}^1, \Delta_{n-1}^2, \mathcal{C}(y_n)),$$

each carrying its two primal blocks, the image u (one channel) and the auxiliary field w (two channels), i.e. three channels per quantity. This gives $7 \times 3 = 21$ input channels. Stacking the iterates together with the data-fidelity gradient $\mathcal{C}(y_n) = K^\top(Ky_n - y)$ gives the network direct access both to the primal variables and to the local residual that drives them. The previous deviations $(\Delta_{n-1}^1, \Delta_{n-1}^2)$ give the network a short memory of the trajectory: the deviation at step n is predicted afresh from the state, but informed by the one at step $n - 1$ rather than chosen in isolation.

We also experimented with giving the network an explicit time embedding of the iteration index n , so that it could in principle adapt its correction to the stage of the trajectory. In

practice this did not prove very effective, it brought no measurable improvement in convergence, so we removed it to keep the architecture simple. The same weights are therefore applied at every step, with the short deviation memory above as the only trace of where in the trajectory the network is acting.

Backbone. The network is a deliberately lightweight, fully feed-forward convolutional stack modelled on the `convnet` of [2]. The 21 input channels are first passed through an Instance Normalisation layer [15], and the body then applies $B = 2$ identical layers of the form

$$h \mapsto \text{Conv}_{3 \times 3} \rightarrow \text{IN} \rightarrow \text{LReLU},$$

each with $H = 32$ feature channels, and a final 3×3 convolution projects onto the $2 \times 3 = 6$ output channels, split into the primal deviations h_n^1 and h_n^2 (the dual blocks (p, q) receive zero deviation). We use 3×3 kernels because they are the smallest stencil that resolves orientation, and stacking three of them accumulates an effective receptive field of about 7×7 pixels (enough to capture the local structure of a CT slice without any fully connected layer). The network has only about 1.7×10^4 parameters, matching the shallow depth of the reference `convnet`.

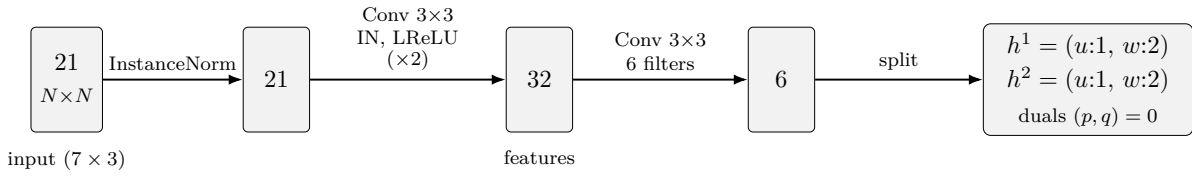


Figure 5.1: The deviation network $\mathcal{N}_\theta$. The algorithm state (seven quantities, three channels each, hence 21 input channels) is passed through Instance Normalisation, two $\text{Conv}_{3 \times 3}$ -IN-LReLU blocks at 32 channels, and a final $\text{Conv}_{3 \times 3}$ to 6 channels, split into the two primal deviations h_n^1, h_n^2 (the dual blocks receive zero deviation). The spatial size $N \times N$ is preserved throughout (padding one); only the channel count changes, $21 \rightarrow 32 \rightarrow 32 \rightarrow 6$.

Activations and normalisation. The nonlinearity is LeakyReLU with negative slope 0.01, which avoids dead units while keeping the map almost linear. Instance Normalisation [15] is applied to the raw input and after every convolution. Rescaling each feature map per image stabilises training when the input channels live on very different scales (the iterates lie in $[0, 1]$ whereas $\mathcal{C}(y_n)$ varies by orders of magnitude with the noise level and the iteration).

5.3 From raw output to a safeguarded deviation

The raw output (h^1, h^2) is first normalised to the unit sphere and then rescaled by the safeguarding scale

$$s_n = \alpha_{\text{safe}} \sqrt{\frac{\zeta \ell_n}{c_u \|h_n^1\|^2 + c_v \|h_n^2\|^2}}, \quad (5.3)$$

where $\alpha_{\text{safe}} \in (0, 1)$ is a strict contraction factor and c_u, c_v are the parameter-dependent coefficients appearing in (4.2). By construction $s_n h_n^i$ satisfies the safeguarding constraint (4.2),

so Theorem 4.3 applies regardless of the network output. In words: the network chooses the direction, the safeguard chooses the largest magnitude that is still provably safe.

5.4 Training

Data. Training instances are synthetic TGV phantoms generated with ODL [16]. Each instance is drawn from a distinct random seed: a phantom $u^\star$ is sampled, the forward model $y = Ku^\star + \eta$ is formed, and a relative noise level is drawn at random so that the network sees a whole range of noise regimes rather than a single setting. The seeds are split into disjoint training and test sets, so the reported numbers reflect generalisation to unseen phantom and noise realisations. The concrete grid sizes, geometry, noise range and seed counts are collected in Chapter 6.

Unrolling and loss. The model unrolls the safeguarded forward–backward iteration for a finite horizon, the deviation network sharing its weights across all steps (the same network is reused at every step), and the training loss is evaluated at the final unrolled iterate, so that the network is optimised for the state the iteration actually reaches. To prevent it from overfitting one fixed horizon, the number of unrolled steps is drawn at random within a range at each training pass rather than being held fixed. This randomised-horizon strategy follows [2]. Which quantity to place under this loss (the monotone-inclusion residual or the TGV² objective value) is itself a design question, and we study it empirically in Chapter 6. Optimisation uses AdamW with a cosine learning-rate schedule.

Chapter 6

Experiments and Results

6.1 Experimental setup

The experiment was conducted on 128×128 CT, starting from an initial state x_0 which we chose arbitrarily to be zero. What we really want to measure is what the scheme can achieve in about *ten iterations*. This is the regime we care about, both because it is the window the network was trained on and because, in practice, a reconstruction solver is only useful if it reaches a good image within a small, affordable number of iterations. Ten iterations is therefore the point at which we compare the methods. We then let each iteration run for about a thousand steps, not because those late steps matter in practice, but only to observe the long-term behaviour and

make sure the scheme stays convergent far beyond the horizon it was trained on: how does it behave on a state it has never seen before?

Throughout the chapter we compare three versions of the *same* safeguarded iteration, which differ only in how the deviations (Δ_n^1, Δ_n^2) are produced.

- **Zero deviations** set $\Delta_n^1 = \Delta_n^2 = 0$, recovering the plain forward–backward scheme. It is both the reference we want to beat and an acceptance test: with the deviations switched off the iteration must still converge to the correct image, which validates the whole implementation.
- **Learned deviations** are the directions chosen by the trained network. This is the method we propose.
- **Random deviations** replace the network output by random directions of the same safeguarded magnitude. This is a control that isolates whether an acceleration comes from learning a good direction, or merely from perturbing the iteration with any nonzero deviation.

Comparing the three answers the two questions that matter: does learning help (learned versus zero), and is the gain specific to learning rather than to perturbation (learned versus random)? Because all three share the same safeguard, none of them can diverge, so the comparison is entirely about speed.

The configuration was not tuned from scratch. As explained before our reference point throughout is the forward–backward-with-deviations framework of Banert et al. [1]: the parameter sequences, the admissible step-size regime, the safeguard budget and the way convergence is measured all come from that work. The only quantities we set ourselves are the ones specific to the imaging problem (the discretisation grid, the geometry of the forward operator, the regularisation weights and the noise range) and even these were chosen to keep the setting comparable to the experiments reported there. This configuration allowed us to see any acceleration we observe as an effect of the learned deviations rather than of favourable but arbitrary tuning.

Problem instance. Each instance is a synthetic TGV phantom generated with ODL [16]: a piecewise-smooth image $u^\star$, its sinogram $y = Ku^\star + \eta$ under the forward operator, and the back-projection $K^\top y$ used to initialise the iteration. Figure 6.1 shows the three. The back-projection is blurred and contaminated by the null space of K . It is precisely the ill-posedness that the TGV² regularisation is meant to undo, and the starting point from which every scheme below must recover $u^\star$.

Discretisation and geometry. The network is trained on 128×128 images (its transfer to 512×512 is studied separately in Section 6.6). The Radon transform is sampled over a full rotation: 1000 equally spaced projection angles in $[0, 2\pi)$ and a linear detector of 1000 cells. We normalise K by power iteration so that $\|K\| = 1$, which (as Section 6.2 explains) is what restores a step size of order one. The relative noise level is drawn at random in $[0, 0.1]$ at each instance, so the network sees a range of noise regimes rather than a single one. We use 200 training seeds and 40 disjoint test seeds, and optimise for 70 epochs with AdamW (learning rate 10^{-3}) under a cosine learning-rate schedule. We fix the safeguard fraction $\alpha_{\text{safe}} = 0.99$.

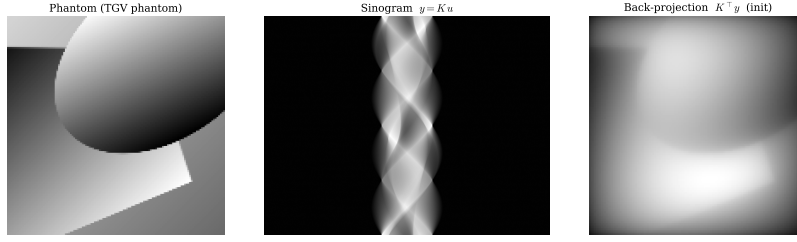


Figure 6.1: Left: the ground-truth TGV phantom u^* . Centre: its sinogram $y = Ku^*$. Right: the back-projection $K^\top y$ used to initialise the iteration. The blur and streaking of $K^\top y$ illustrate the ill-posedness the reconstruction has to overcome.

Parameter sequences. Following [1] we take

$$\lambda_n = \lambda_0(1+n)^e \text{ with } \lambda_0 = 0.9, \ e = 0.3, \quad \mu_n = \frac{1}{\lambda_0} \lambda_n^2 - \lambda_n.$$

The exponent $e \in [0, 1]$ interpolates between standard forward–backward splitting ($e = 0$) and the accelerated regime whose theory promises an $O(1/n^2)$ rate ($e = 1$). The assumptions of [1] forbid faster-than-linear growth of λ_n , so $e = 1$ is the theoretical optimum. We had therefore expected $e = 1$ to converge fastest. However in [1] the fastest convergence was obtained for $e = 0.3$ and a sweep over several e confirmed this choice. We fix $e = 0.3$ throughout. With this choice all the parameter conditions of Assumption 4.1 are satisfied.

Taking $\bar{\beta} = \beta = 1$, condition (2) of Assumption 4.1 yields the step bound $0 \leq \gamma \leq 2.2$. We fix $\zeta_n = 0.9$ constant: taking ζ large gives the deviations maximum amplitude to converge faster. This is not necessarily globally optimal, a smaller deviation that enables a larger downstream step could in principle be more useful than two medium steps, but optimising the budget schedule itself is left to future work.

Two residuals to be studied. Convergence can be measured in two natural ways, and we refer to them throughout the report. The first is the residual of the monotone inclusion, $\|(\mathcal{A} + \mathcal{C})(x_n)\|$, which we call the *KKT residual*: it vanishes exactly at a solution of (3.5), so its decay measures progress towards the fixed point. The second is the *objective residual*, the TGV² objective gap $f(x_n) - f^*$ (where $f(x)$ is the function in the initial problem $\min_x f(x)$ and f^* its optimum). It is interesting because it lets us compare our scheme with known methods such as PDHG, which minimise the same objective. Which of the two is the better training target is itself a question, studied experimentally in Section 6.3. In the two following sections we still need to fix a few parameters, and we do so based on the KKT residual, as it was the first and most natural quantity we used.

Regularisation weights. The weights α_1, α_2 fix the image the iteration converges to. We first took $\alpha_1 = \alpha_2 = 0.4$ (following ODL examples [16]): the learned phase is healthy and the acceleration is clearly visible over the first ten iterations, but the reconstruction stays over-smoothed (≈ 23 dB), because weights this large over-penalise the gradient. A two-dimensional sweep scoring the reconstruction PSNR singled out $\alpha_1 = 0.004$, $\alpha_2 = 0.04$ as the sharpest TGV² image (the gap $\alpha_2 = 10\alpha_1$ lets the second-order term smooth flat regions while the

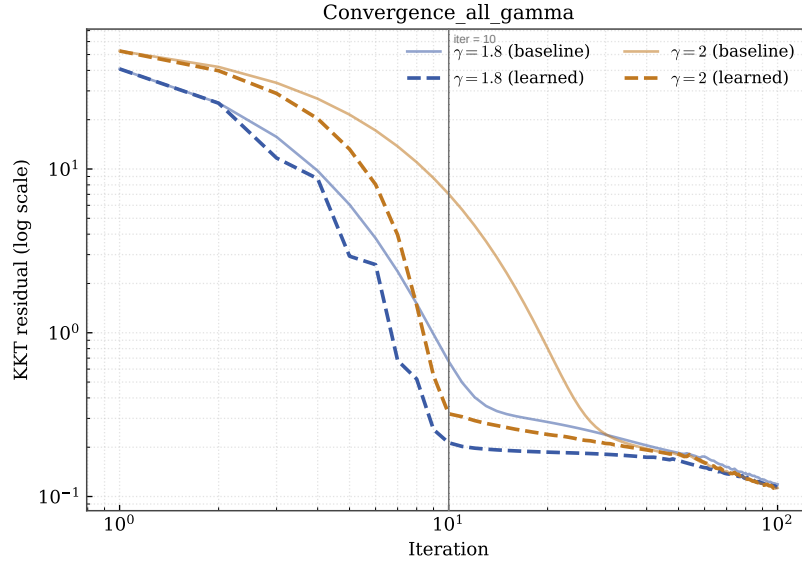
first-order weight preserves edges), and we use these throughout.

Remark 6.1 (Peak signal-to-noise ratio (PSNR)). The PSNR is a standard measure of reconstruction quality, expressed in decibels. For a reconstruction u compared with a reference image u^* on an $N \times N$ grid, it is built from the mean squared error $\text{MSE} = \frac{1}{N^2} \|u - u^*\|^2$ by

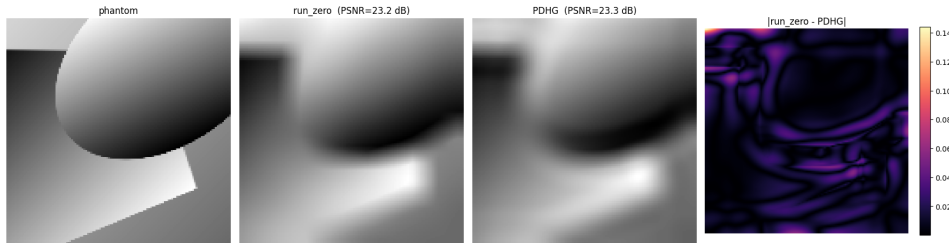
$$\text{PSNR}(u, u^*) = 10 \log_{10} \left(\frac{u_{\max}^2}{\text{MSE}} \right),$$

where $u_{\max}$ is the maximal admissible pixel value (here $u_{\max} = 1$, since our images are scaled to $[0, 1]$). A smaller error gives a larger PSNR, so higher is better, and each extra 10 dB corresponds to a tenfold reduction of the mean squared error. It is the metric we use to compare reconstructions at a fixed iteration count.

Here u^* is obtained by running the reference algorithm (PDHG) for many iterations (5000), giving the reference image the reconstruction should approach.



(a) KKT residual (plateaus at 10^{-1}).



(b) Reconstruction (≈ 23 dB, over-smoothed).

Figure 6.2: Effect of the regularisation weights at $\alpha_1 = \alpha_2 = 0.4$. (a) the learned iteration (dashed) still accelerates over its baseline but everything plateaus. (b) the converged image is blurry, far from the ≈ 37 dB result at the swept weights $\alpha_1 = 0.004$, $\alpha_2 = 0.04$. This motivated the 2D sweep.

6.2 Step-size selection on the baseline

The first attempt at tomography did not converge, and the cause was the step size, not the algorithm. The data gradient $\mathcal{C}$ has cocoercivity bound $\bar{\beta} = \|K\|^2$. For denoising $\|K\| = 1$, but the raw Radon transform has a large operator norm, so the outer condition $\gamma\bar{\beta} \leq 4 - 2\lambda_0$ collapses γ to about 10^{-3} and the data update barely moves the iterate. Renormalising K to unit norm restores $\bar{\beta} = 1$ and a step of order one. Together with the γ -independent FISTA resolvent (Section 4.3), this makes tomography behave like denoising. Figure 6.3 then sweeps $\gamma \in (0, 2.2)$ on the zero-deviation scheme: convergence holds throughout but is *not* monotone in γ , fastest around $\gamma \approx 1.5$ – 1.8 and slowing near the bound, while $\gamma = 2.5$ (outside the range) diverges, confirming the bound.

We fix $\gamma = 1.8$: at the fast end of this sweet spot the baseline is about as fast as the scheme ever gets, so it is the *hardest* step to improve upon, which makes an acceleration there the most convincing test. Some plots use a larger $\gamma = 2$, where the wider margin makes the acceleration easier to see.

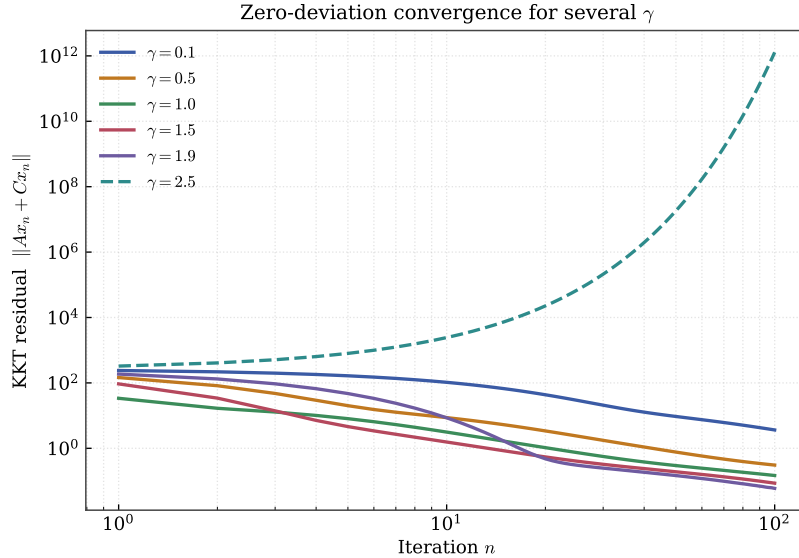


Figure 6.3: KKT residual $\|(\mathcal{A} + \mathcal{C})(x_n)\|$ versus iteration for the zero-deviation iteration under several step sizes γ . The scheme converges across the whole admissible range $(0, 2.2)$, but not monotonically in γ : the decay is fastest around $\gamma \approx 1.5$ – 1.8 and slows again for the largest admissible steps, while $\gamma = 2.5$ (outside the bound) diverges. This motivates the choice $\gamma = 1.8$, at the fast end of the range, used throughout.

6.3 Choosing the training target: inclusion residual versus objective

As explained earlier, there are two natural quantities to place under the unrolled network at training time. The first is the monotone-inclusion residual, which we call the *KKT residual*,

$$\mathcal{L}(\theta) = \|(\mathcal{A} + \mathcal{C})(x_T)\|, \quad (6.1)$$

which vanishes exactly at a solution of the inclusion (3.5) and penalises every block of the optimality system at once, primal and dual. The second is the TGV² *objective value* $f(x_T)$ itself, the classical optimisation target, a function of the primal variables only.

The training strategy is the same for both. We unroll the scheme for a short horizon (about ten iterations), evaluate the chosen quantity at the final iterate x_T , and back-propagate through the unrolled iteration to adjust the network weights θ so as to make that quantity as small as possible. The loss is therefore exactly the residual we choose to monitor.

What we ultimately care about is the optimisation problem $\min_u f(u)$: we want the reconstruction to reach an objective value $f(x_T)$ as close to the optimum f^* as possible.

6.3.1 Training on the KKT residual

Figure 6.4a compares three iterations at $\gamma = 1.8$, evaluated on the residual (6.1): the learned deviations, the zero-deviation baseline, and some random directions (in the convergence set). All three are run for 10^3 test iterations, although the network is unrolled for only a short horizon of 10 to 20 steps during training.

The zero-deviation curve is our reference, the one we want to beat. As noted in [1], setting the deviations to zero makes the scheme fall back on the standard, unaccelerated forward–backward iteration. The random deviations, by contrast, are the worst thing we can do: they follow no strategy at all on where to go.

With the deviations switched off the iteration must still converge to the correct solution, and it does. Against this baseline the learned deviations reduce the residual markedly faster in the early regime. The gain is attributed to learning, as the random deviations confirm that converging fast requires making the right choice of deviations. Finally, the safeguard (Section 4.2) guarantees that the learned curve never diverges: it stays convergent over the full 10^3 -iteration horizon, roughly two orders of magnitude beyond the training window. The acceleration is thus obtained without sacrificing the asymptotic guarantee of the underlying scheme.

The acceleration is also robust to the step size, not specific to $\gamma = 1.8$ (fig. 6.4b).

6.3.2 Does it also lower the objective? Comparison with PDHG

A residual trained to be small must still correspond to genuine progress on the problem we care about. We therefore check the network (trained on the KKT residual) on the TGV² objective gap $f(x_n) - f^*$, which also lets us compare against the standard solver for this problem, the Chambolle–Pock algorithm (PDHG) from ODL [16, 7]. f^* is obtained by running PDHG to high accuracy (5000 iterations). We read the comparison in the early regime that matters, since the network is trained on a horizon of only about ten steps, comparing at a thousand iterations would be meaningless. Figure 6.5a shows that there the residual-trained network drives the objective gap down far faster than PDHG: after ten iterations its gap is already around 10^{-1} while PDHG is still near 10^2 . So optimising the inclusion residual does lower the objective, and does so much faster than the reference solver, by essentially the same early-regime margin as the network trained directly on the objective (fig. 6.7a).

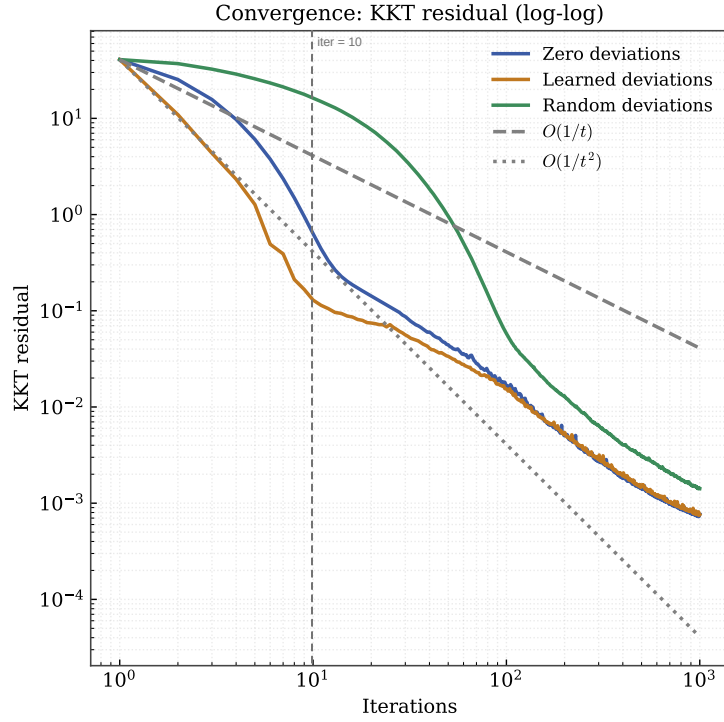
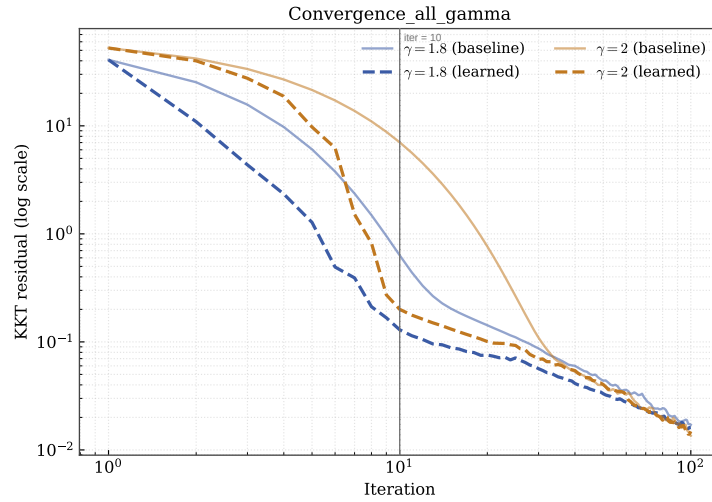
(a) Learned vs zero vs random ($\gamma = 1.8$, 1000 it.).(b) Robustness across $\gamma \in \{1.8, 2\}$.

Figure 6.4: KKT residual $\|(\mathcal{A} + \mathcal{C})(x_n)\|$ versus iteration (log-log). **(a)** Learned deviations (orange) track the $O(1/t^2)$ reference in the early regime, beating the zero-deviation baseline (blue) and random directions (green), and stay convergent over the full horizon despite the 10–20 step training window. **(b)** The learned scheme (dashed) accelerates the early decay over its baseline (solid) at both $\gamma = 1.8$ and $\gamma = 2$.

The same holds for the other variants: learning beats every baseline, and a learned iteration run without the safeguard diverges past the training horizon (a result already observed in [2]), which isolates the role of the safeguard.

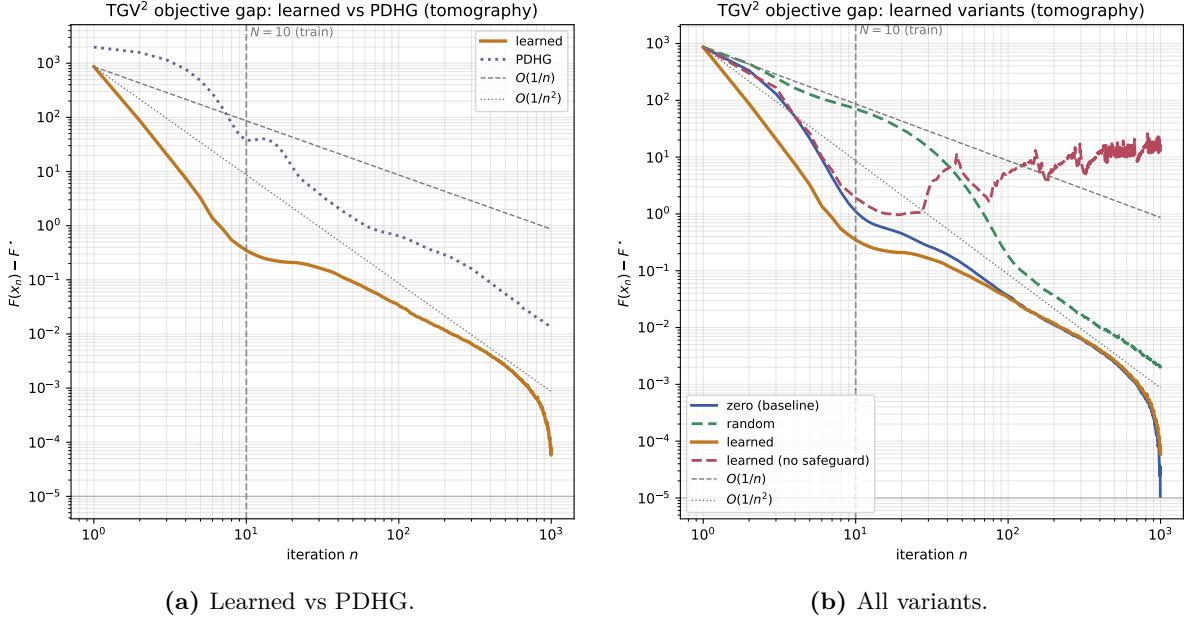


Figure 6.5: TGV² objective gap $f(x_n) - f^*$ versus iteration (log-log), for the *residual*-trained network. (a) The learned scheme lowers the objective far faster than PDHG over the full horizon. (b) It is the lowest of all variants; the unsafeguarded one diverges.

Finally, the early-regime distance to the optimal image confirms the same ordering on the quantity that matters most in practice. Figure 6.6 plots the relative distance $\|u_n - u^*\|$ to the PDHG reference over the first 10 iterations: the learned iteration reaches any prescribed accuracy first, the zero-deviation and random iterations follow, and PDHG is slowest in this early regime. The early residual acceleration therefore translates directly into faster convergence to the true image in this early regime that we are interested in.

6.3.3 Training on the objective directly

Having validated the residual-trained network, the natural next question is whether training directly on the objective value $f(x_T)$, aligning the loss with the energy rather than with the full optimality system, changes anything. We retrained the same network under this loss and re-ran the same comparisons.

The objective-trained network behaves qualitatively like the residual-trained one. Figure 6.7a shows that it drives the objective gap $f(x_n) - f^*$ far below PDHG over 10 iterations, reaching $\approx 10^{-1}$ where PDHG is still near 10^2 and fig. 6.7b shows it sitting below both the zero-deviation baseline and the random control, with the unsafeguarded variant once more diverging. In other words, optimising the energy directly also yields a safeguarded, accelerated scheme: both training targets are effective.

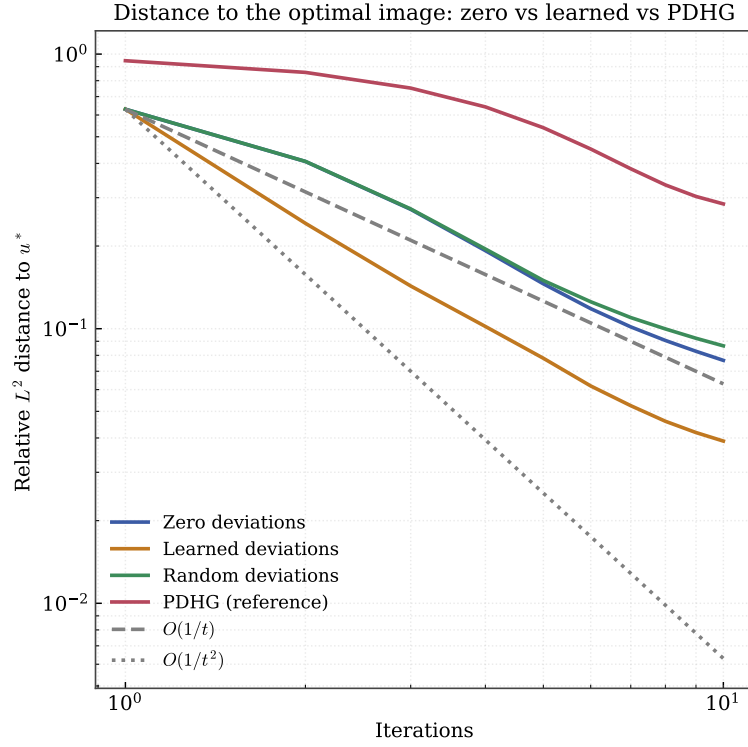


Figure 6.6: Relative L^2 distance $\|u_n - u^*\|$ to the high-accuracy PDHG reference u^* over the first 10 iterations, for the learned, zero-deviation and random iterations and for PDHG. The learned iteration is fastest to approach u^* .

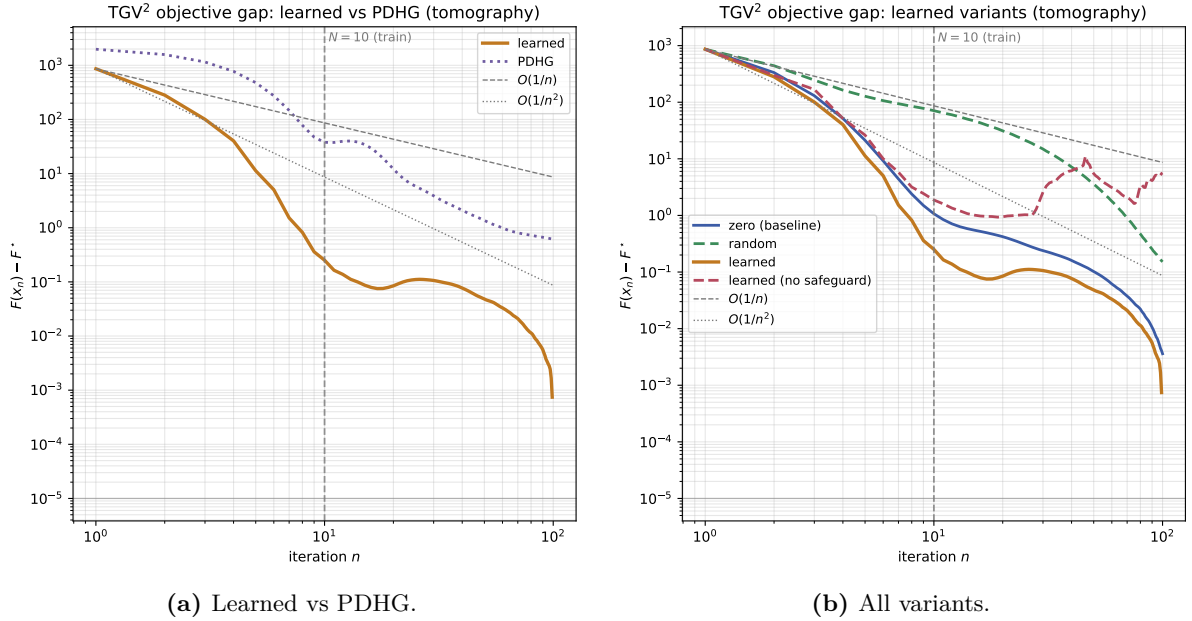


Figure 6.7: Network trained *on the objective value*, TGV^2 objective gap $f(x_n) - f^*$ versus iteration. (a) It lowers the energy markedly faster than PDHG, mirroring the residual-trained result. (b) It sits below the zero-deviation and random iterations; the unsafeguarded variant again diverges.

6.4 Reconstruction quality and PSNR

We next ask whether the residual acceleration carries over to the downstream imaging metric. Having a good-quality image is essential, as it is the main application of our problem. A convergent residual is not automatically a good image: an iteration can drive the residual down while the reconstruction barely improves, which is exactly why we track the imaging metric directly.

Figure 6.8 tracks the reconstruction PSNR on the held-out test set over the first iterations. The learned deviations climb fastest, reaching about 30 dB within ten iterations, the zero-deviation baseline is slower, and PDHG slower still in this early regime. The acceleration thus reaches the visually relevant quality in a handful of iterations.

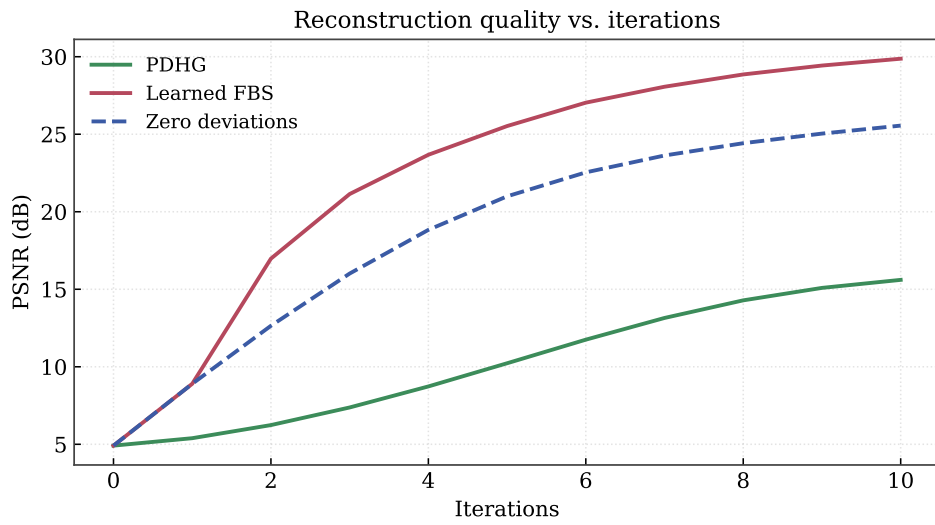


Figure 6.8: Reconstruction PSNR (dB) on the held-out test set versus iteration. Learned FBS (red) climbs fastest, reaching about 30 dB within ten iterations; the zero-deviation baseline (dashed blue) is slower and PDHG (green) slower still in the early regime.

After only ten iterations the visual gap is clear (fig. 6.9): the learned iterate (≈ 30 dB) renders the object boundary and flat interior cleanly, the zero-deviation iterate (≈ 26 dB) is softer, and PDHG (≈ 16 dB) is still dominated by back-projection blur.

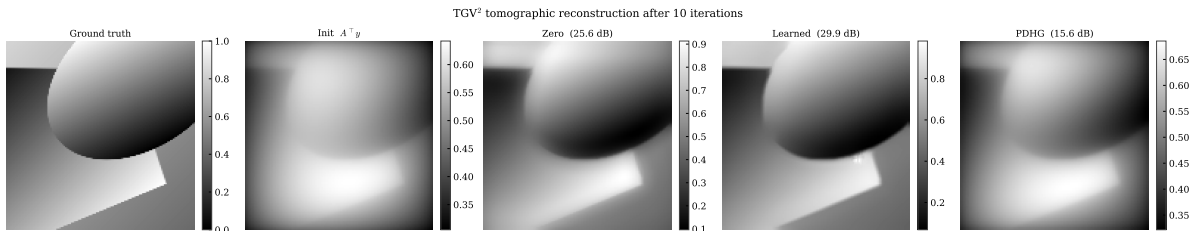


Figure 6.9: Reconstructions after 10 iterations. From left to right: ground truth, the back-projection initialisation $K^\top y$, the zero-deviation iterate (≈ 26 dB), the learned iterate (≈ 30 dB) and PDHG (≈ 16 dB). After the same ten iterations the learned reconstruction is the sharpest.

Run to convergence, all schemes reach the same high-quality image (fig. 6.10): at 1000 iterations the zero-deviation reconstruction (≈ 36.8 dB) matches the independent PDHG

reference (≈ 36.6 dB), indistinguishable to the eye, the visual counterpart of the acceptance test.

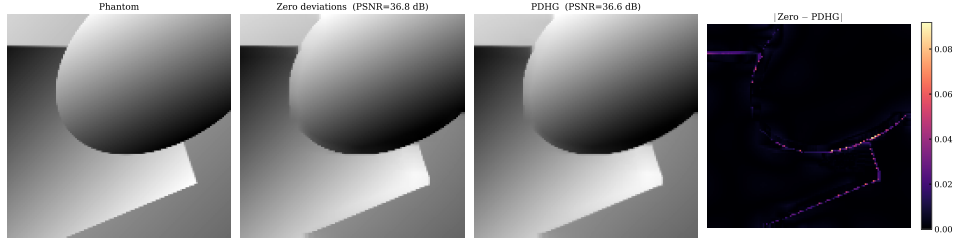


Figure 6.10: Reconstruction at convergence. From left to right: the TGV phantom, the zero-deviation reconstruction ($\text{PSNR} \approx 36.8$ dB), the independent PDHG reference ($\text{PSNR} \approx 36.6$ dB) and the difference image $|\text{Zero} - \text{PDHG}|$. The two solvers converge to the same image.

6.5 Training behaviour

Figure 6.11 reports the training and validation loss (6.1) over the training run at $\gamma = 1.8$. The validation error decreases steadily and tracks the training error. This is expected because the random unrolling horizon makes each training pass a slightly different (and on average harder) problem than a fixed-horizon evaluation. The absence of a widening generalisation gap, together with the fact that test seeds are disjoint from training seeds, indicates that the network learns a genuinely transferable steering rule rather than memorising specific phantoms.

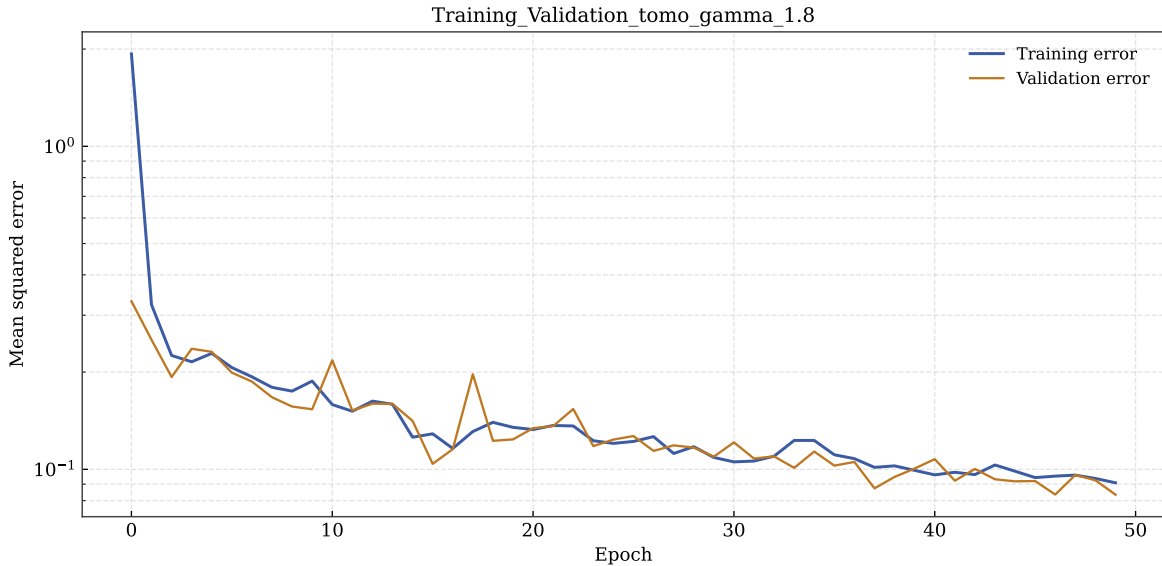


Figure 6.11: Training and validation error (mean inclusion residual) versus epoch at $\gamma = 1.8$: the validation curve tracks the training curve, indicating stable optimisation and good generalisation to unseen phantom/noise realisations.

6.6 Generalisation across resolution

Because the deviation network is fully convolutional, with no fully connected layer, the same weights act at every pixel, so it carries no dependence on the image size and can be evaluated at a resolution different from the one it was trained on. Figure 6.12 confirms this: a network trained exclusively on 128×128 problems is applied, without any retraining, to 512×512 problems, where it still accelerates the early decay over the zero-deviation baseline and again leaves the random-direction control behind. This matters in practice (training on small problems is cheap whereas deployment targets full-resolution scans) and the lightweight convnet of Chapter 5 was chosen partly to make 512×512 unrolling fit in memory. The learned steering rule therefore transfers across resolutions without retraining.

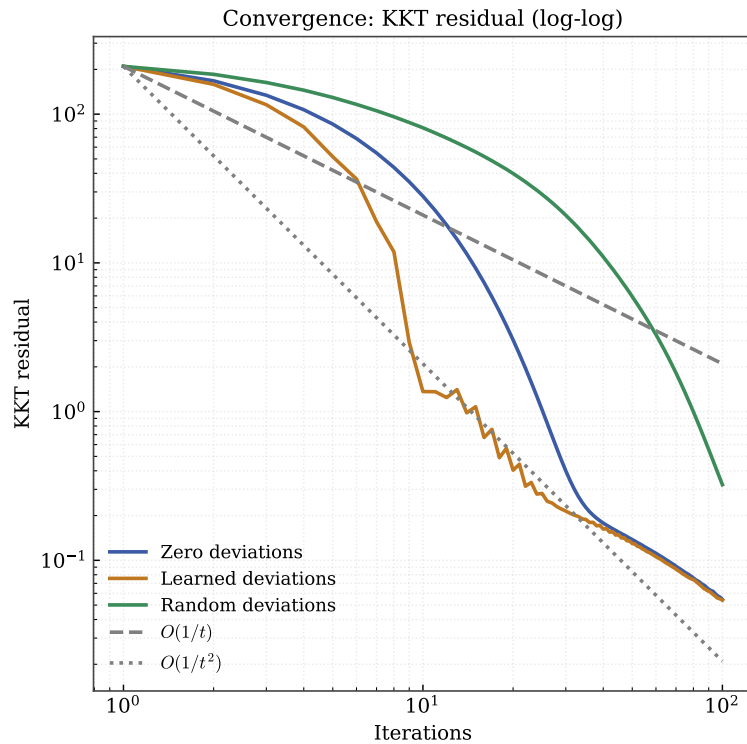


Figure 6.12: Cross-resolution generalisation. A network trained only on 128×128 , applied without retraining to 512×512 , still decays faster than the zero-deviation baseline (blue) and the random control (green), tracking the $O(1/t^2)$ slope in the early regime.

6.7 Discussion

The experiments support a consistent picture: the learned deviations bend the residual curve well below the $O(1/t)$ slope and, over the first iterations, even below the $O(1/t^2)$ reference, before settling onto the baseline asymptotics. PDHG (Chambolle–Pock) serves both as the high-accuracy reference u^* and as a competing solver, and on both the distance-to-reference and the PSNR the learned scheme reaches a given accuracy in markedly fewer iterations. Three controls make the claim robust: the random-direction baseline rules out that the gain is mere perturbation, the distance and PSNR curves rule out that the residual is gamed without solving

the problem, and the 1000-iteration horizon (with the divergence of the unsafeguarded variant) rules out late-stage divergence. The cross-resolution transfer is the most practically significant result.

The main limitation is that the budget schedule ζ_n is fixed and that the gains, clear in the early regime, do not change the asymptotic rate (by design, since the safeguard ties the learned scheme to the baseline’s guarantees). Whether a network that also shapes the budget could improve the asymptotics without breaking Theorem 4.3 is an open question.

Chapter 7

Conclusion and Perspectives

7.1 Summary

This internship accelerated TGV² tomographic reconstruction by learning the deviations of a convergence-safeguarded forward–backward scheme. We recast the problem as a monotone inclusion $0 \in \mathcal{A}x + \mathcal{C}x$, solved its resolvent by FISTA, and trained a lightweight convolutional network to predict the deviations, with a safeguard layer keeping the iteration provably convergent whatever the network outputs. Empirically the learned deviations reach a given accuracy in far fewer iterations than both the zero-deviation baseline and PDHG (on the KKT residual, the objective gap and the PSNR alike): a random-direction control attributes the gain to learning, the safeguard preserves convergence far beyond the training horizon, and the fully convolutional network transfers from 128×128 to 512×512 without retraining.

7.2 Perspectives

Several directions extend naturally from this work.

Learning the budget and the algorithm sequences. Here the network learned only the direction of the deviations, while the budget fraction ζ_n and the parameter sequences (γ_n, λ_n) were kept constant. Letting the network also allocate the budget across iterations (little early on, more once the iteration is well oriented) and adapt the step sizes could help the convergence even more.

Reducing the cost of the resolvent. Every outer iteration solves the resolvent of $\mathcal{A}$ with an inner FISTA loop of about a hundred iterations, which dominates the cost. Learning a cheap

approximate resolvent, or unrolling only a few inner steps, would cut the number of forward and back projections needed to reach a given accuracy, the quantity that matters in CT.

Beyond synthetic phantoms. Our training set is not optimal: it is built from synthetic TGV phantoms with artificial noise rather than real scans. Training on real CT data with clinically realistic noise and dose regimes, like the Mayo data set, would help obtain a more robust trained model, but I did not have time to look into real datasets.

Bibliography

- [1] Hamed Sadeghi, Sebastian Banert, and Pontus Giselsson. Incorporating history and deviations in forward–backward splitting. *Numerical Algorithms*, 96:1819–1867, 2024.
- [2] Sebastian Banert, Jevgenija Rudzusika, Ozan Öktem, and Jonas Adler. Accelerated forward-backward optimization using deep learning. *arXiv preprint arXiv:2105.05210*, 2021.
- [3] Frank Natterer. *The Mathematics of Computerized Tomography*. SIAM, 2001.
- [4] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D: Nonlinear Phenomena*, 60(1-4):259–268, 1992.
- [5] Kristian Bredies, Karl Kunisch, and Thomas Pock. Total generalized variation. *SIAM Journal on Imaging Sciences*, 3(3):492–526, 2010.
- [6] Kristian Bredies. Recovering piecewise smooth multichannel images by minimization of convex functionals with total generalized variation penalty. In *Efficient Algorithms for Global Optimization Methods in Computer Vision*, volume 8293 of *Lecture Notes in Computer Science*, pages 44–77. Springer, 2014. ISBN 978-3-642-54774-4.
- [7] Antonin Chambolle and Thomas Pock. A first-order primal-dual algorithm for convex problems with applications to imaging. *Journal of Mathematical Imaging and Vision*, 40(1):120–145, 2011.
- [8] Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009.
- [9] Hangqi Wu, Pengcheng Zhang, Shu Li, Jing Lu, Yi Liu, Jiaqi Kang, and Zhiguo Gui. Low-dose CT reconstruction network based on the unfoldment of second-order TGV. *Journal of X-Ray Science and Technology*, 2026.
- [10] Vishal Monga, Yuelong Li, and Yonina C. Eldar. Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing. *IEEE Signal Processing Magazine*, 38(2):18–44, 2021.
- [11] Heinz H. Bauschke and Patrick L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. Springer, 2nd edition, 2017.

- [12] Andrea Simonetto. Continuous optimisation: Theory and algorithms, part i: Theory, 2025. Lecture notes 4OPT1, ENSTA Paris, Institut Polytechnique de Paris, academic year 2025/2026.
- [13] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [14] Gregory Ongie, Ajil Jalal, Christopher A. Metzler, Richard G. Baraniuk, Alexandros G. Dimakis, and Rebecca Willett. Deep learning techniques for inverse problems in imaging. *IEEE Journal on Selected Areas in Information Theory*, 1(1):39–56, 2020.
- [15] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv preprint arXiv:1607.08022*, 2016.
- [16] Jonas Adler, Ozan Öktem, et al. ODL: Operator discretization library. <https://github.com/odlgroup/odl>, 2017.